Announcement

- SRE 3 next week - likely in the beginning of class. Bring your laptops for SRE reflection.
- Pset 6 due on Nov 12, so enough time to work on!

Propositional Logic

Logic describes digital circuits and lets us encode computational problems.

Definition: Boolean formula
A formula built from variables $x_0, \ldots, x_{n-1}$, parentheses (,), and

$$\wedge \text{ (AND)}, \qquad \vee \text{ (OR)}, \qquad \neg \text{ (NOT)}.$$

Propositional Logic

Logic describes digital circuits and lets us encode computational problems.

Definition: Boolean formula
A formula built from variables $x_0, \ldots, x_{n-1}$, parentheses (,), and

$$\wedge \text{ (AND)}, \qquad \vee \text{ (OR)}, \qquad \neg \text{ (NOT)}.$$

Every formula defines a Boolean function

$$\varphi : \{0,1\}^n \longrightarrow \{0,1\}, \qquad 0 = \text{false, unsatisfied}, \quad 1 = \text{true, satisfied}.$$

Boolean Formulas: Examples

$$\varphi_{\mathrm{maj}}(x_0, x_1, x_2) = (x_0 \wedge x_1) \vee (x_1 \wedge x_2) \vee (x_2 \wedge x_0).$$

Q: For which inputs does this formula evaluate to 1?

Boolean Formulas: Examples

$$\varphi_{\mathrm{maj}}(x_0, x_1, x_2) = (x_0 \wedge x_1) \vee (x_1 \wedge x_2) \vee (x_2 \wedge x_0).$$

Q: For which inputs does this formula evaluate to 1?

$$\varphi_{\mathrm{pal}}(x_0, x_1, x_2, x_3) = \big((x_0 \wedge x_3) \vee (\neg x_0 \wedge \neg x_3)\big) \wedge \big((x_1 \wedge x_2) \vee (\neg x_1 \wedge \neg x_2)\big).$$

Q: What does this formula check about the string $x_0x_1x_2x_3$?

DNF and CNF

- A *literal* is a variable x_i or its negation $\neg x_i$.
- A *term* is an AND of literals.
- A *clause* is an OR of literals.

DNF and CNF

- A *literal* is a variable x_i or its negation $\neg x_i$.
- A *term* is an AND of literals.
- A *clause* is an OR of literals.

Disjunctive normal form (DNF)
An OR of terms.

$$(x_0 \wedge \neg x_1) \vee (x_1 \wedge x_2).$$

Conjunctive normal form (CNF)
An AND of clauses.

$$(x_0 \vee \neg x_1) \wedge (x_1 \vee x_2).$$

DNF, CNF, Both, or Neither?

Q: Classify each formula as written.

1. $\varphi_{\mathrm{maj}} = (x_0 \wedge x_1) \vee (x_1 \wedge x_2) \vee (x_2 \wedge x_0)$.
2. $\varphi_{\mathrm{pal}} = \big((x_0 \wedge x_3) \vee (\neg x_0 \wedge \neg x_3)\big)$
   $\wedge \big((x_1 \wedge x_2) \vee (\neg x_1 \wedge \neg x_2)\big)$.
3. $\neg x_0 \vee x_1$.

DNF, CNF, Both, or Neither?

Q: Classify each formula as written.

1. $\varphi_{\mathrm{maj}} = (x_0 \wedge x_1) \vee (x_1 \wedge x_2) \vee (x_2 \wedge x_0)$.
2. $\varphi_{\mathrm{pal}} = \big((x_0 \wedge x_3) \vee (\neg x_0 \wedge \neg x_3)\big)$
   $\wedge \big((x_1 \wedge x_2) \vee (\neg x_1 \wedge \neg x_2)\big)$.
3. $\neg x_0 \vee x_1$.

The last formula $\neg x_0 \vee x_1$ is also known as 'implication': $x_0 \implies x_1$. The idea is that 'true statement must only imply true statement'.

Every Boolean Function Has a DNF and a CNF

Lemma. For every $f : \{0,1\}^n \to \{0,1\}$, there exist a DNF φ and a CNF ψ with $f \equiv \varphi \equiv \psi$. Here $f \equiv g$ means $f(x) = g(x)$ for every input x.

Every Boolean Function Has a DNF and a CNF

Lemma. For every $f : \{0,1\}^n \to \{0,1\}$, there exist a DNF φ and a CNF ψ with $f \equiv \varphi \equiv \psi$. Here $f \equiv g$ means $f(x) = g(x)$ for every input x.

Importance in computing: DNFs and CNFs are made up of simple functions (AND, OR, NOT) - they can still represent an arbitrary function f, however complex.

Every Boolean Function Has a DNF and a CNF

Lemma. For every $f : \{0,1\}^n \to \{0,1\}$, there exist a DNF φ and a CNF ψ with $f \equiv \varphi \equiv \psi$. Here $f \equiv g$ means $f(x) = g(x)$ for every input x.

Importance in computing: DNFs and CNFs are made up of simple functions (AND, OR, NOT) - they can still represent an arbitrary function f, however complex. Caveat?

Every Boolean Function Has a DNF and a CNF

Lemma. For every $f : \{0,1\}^n \to \{0,1\}$, there exist a DNF φ and a CNF ψ with $f \equiv \varphi \equiv \psi$. Here $f \equiv g$ means $f(x) = g(x)$ for every input x.

Importance in computing: DNFs and CNFs are made up of simple functions (AND, OR, NOT) - they can still represent an arbitrary function f, however complex. Caveat?

Truth table: a list $\{(\alpha, f(\alpha)), \forall \alpha \in \{0,1\}^n\}$. Its size is 2^n.

Every Boolean Function Has a DNF and a CNF

Lemma. For every $f : \{0,1\}^n \to \{0,1\}$, there exist a DNF φ and a CNF ψ with $f \equiv \varphi \equiv \psi$. Here $f \equiv g$ means $f(x) = g(x)$ for every input x.

Importance in computing: DNFs and CNFs are made up of simple functions (AND, OR, NOT) - they can still represent an arbitrary function f, however complex. Caveat?

Truth table: a list $\{(\alpha, f(\alpha)), \forall \alpha \in \{0,1\}^n\}$. Its size is 2^n.

α	000	001	010	011	100	101	110	111
$\varphi_{\text{maj}}(\alpha)$	0	0	0	1	0	1	1	1
			0-inputs	1-inputs				

Every Boolean Function Has a DNF and a CNF

DNF construction: Parse the truth table and check if x matches a 1 input.

Every Boolean Function Has a DNF and a CNF

DNF construction: Parse the truth table and check if x matches a 1 input.

$$\varphi(x) = \bigvee_{\alpha:\, f(\alpha)=1} \underbrace{\bigwedge_{i=0}^{n-1} (x_i = \alpha_i)}_{\text{true exactly when } x=\alpha} .$$

$$(x_i = \alpha_i) \quad \text{means} \quad \begin{cases} x_i & \text{if } \alpha_i = 1, \\ \neg x_i & \text{if } \alpha_i = 0. \end{cases}$$

Every Boolean Function Has a DNF and a CNF

DNF construction: Parse the truth table and check if x matches a 1 input.

$$\varphi(x) = \bigvee_{\alpha:\, f(\alpha)=1} \underbrace{\bigwedge_{i=0}^{n-1} (x_i = \alpha_i)}_{\text{true exactly when } x=\alpha} .$$

$$(x_i = \alpha_i) \quad \text{means} \quad \begin{cases} x_i & \text{if } \alpha_i = 1, \\ \neg x_i & \text{if } \alpha_i = 0. \end{cases}$$

CNF construction: Parse the truth table and make sure x does not match a 0 input.

$$\psi(x) = \bigwedge_{\alpha:\, f(\alpha)=0} \underbrace{\bigvee_{i=0}^{n-1} (x_i \neq \alpha_i)}_{\text{false exactly when } x=\alpha} .$$

A DNF for Palindromes

The four satisfying inputs are

$$1111, \qquad 1001, \qquad 0110, \qquad 0000.$$

One term per satisfying input gives

A DNF for Palindromes

The four satisfying inputs are

$$1111, \qquad 1001, \qquad 0110, \qquad 0000.$$

One term per satisfying input gives

$$\begin{aligned}\varphi(x_0, x_1, x_2, x_3) = {} & (x_0 \wedge x_1 \wedge x_2 \wedge x_3) \\ & \vee (x_0 \wedge \neg x_1 \wedge \neg x_2 \wedge x_3) \\ & \vee (\neg x_0 \wedge x_1 \wedge x_2 \wedge \neg x_3) \\ & \vee (\neg x_0 \wedge \neg x_1 \wedge \neg x_2 \wedge \neg x_3).\end{aligned}$$

Note that this formula is larger than the earlier formula for palindromes.

Q: How large can these DNF and CNF constructions be, in general?

Computational Problems in Propositional Logic

Input: a Boolean formula φ on n variables.

Output: an assignment $\alpha \in \{0,1\}^n$ with $\varphi(\alpha) = 1$, or $\perp$ if no such assignment exists.

Problem	**Allowed input formulas**
Satisfiability (SAT)	Any Boolean formula
CNF-SAT	CNF formulas
DNF-SAT	DNF formulas

Computational Problems in Propositional Logic

Input: a Boolean formula φ on n variables.

Output: an assignment $\alpha \in \{0,1\}^n$ with $\varphi(\alpha) = 1$, or $\perp$ if no such assignment exists.

Problem	**Allowed input formulas**
Satisfiability (SAT)	Any Boolean formula
CNF-SAT	CNF formulas
DNF-SAT	DNF formulas

Q: Which problem is easy to solve?

DNF-SAT

Satisfying *one term* satisfies the entire DNF.

DNF-SAT

Satisfying *one term* satisfies the entire DNF.

1. Find a term avoiding contradiction (i.e., $x_i, \neg x_i$ in the same term).
2. Set its positive literals to 1 and its negative literals to 0.
3. Set the remaining variables arbitrarily.

If every term contains a contradiction, return $\perp$.

DNF-SAT

Satisfying *one term* satisfies the entire DNF.

1. Find a term avoiding contradiction (i.e., $x_i, \neg x_i$ in the same term).
2. Set its positive literals to 1 and its negative literals to 0.
3. Set the remaining variables arbitrarily.

If every term contains a contradiction, return $\bot$.
Example:

$$(x_0 \wedge \neg x_0) \vee \underbrace{(\neg x_1 \wedge x_2)} \vee (x_2 \wedge \neg x_2).$$

DNF-SAT

Satisfying *one term* satisfies the entire DNF.

1. Find a term avoiding contradiction (i.e., $x_i, \neg x_i$ in the same term).
2. Set its positive literals to 1 and its negative literals to 0.
3. Set the remaining variables arbitrarily.

If every term contains a contradiction, return $\perp$.
Example:

$$(x_0 \wedge \neg x_0) \vee \underbrace{(\neg x_1 \wedge x_2)}_{\alpha_1=0,\ \alpha_2=1} \vee (x_2 \wedge \neg x_2).$$

DNF-SAT

Satisfying *one term* satisfies the entire DNF.

1. Find a term avoiding contradiction (i.e., $x_i, \neg x_i$ in the same term).
2. Set its positive literals to 1 and its negative literals to 0.
3. Set the remaining variables arbitrarily.

If every term contains a contradiction, return $\perp$.
Example:

$$(x_0 \wedge \neg x_0) \vee \underbrace{(\neg x_1 \wedge x_2)}_{\alpha_1=0,\ \alpha_2=1} \vee (x_2 \wedge \neg x_2).$$

Q: Why does the same strategy fail for CNF-SAT?

Graph Coloring as a SAT Problem

Input: a graph $G = (V, E)$ and a number of colors $k \geq 1$.
Goal: assign colors so that adjacent vertices have different colors.

Theorem

Graph Coloring with k colors on n vertices and m edges reduces in $O(n + km)$ time to CNF-SAT with at most kn variables and $n + km$ clauses.

Graph Coloring as a SAT Problem

Input: a graph $G = (V, E)$ and a number of colors $k \geq 1$.
Goal: assign colors so that adjacent vertices have different colors.

Theorem

Graph Coloring with k colors on n vertices and m edges reduces in $O(n + km)$ time to CNF-SAT with at most kn variables and $n + km$ clauses.

Idea: introduce variables to represent boolean statements about colors, and clauses to enforce conditions.

Graph Coloring as a SAT Problem

Input: a graph $G = (V, E)$ and a number of colors $k \geq 1$.
Goal: assign colors so that adjacent vertices have different colors.

Theorem

Graph Coloring with k colors on n vertices and m edges reduces in $O(n + km)$ time to CNF-SAT with at most kn variables and $n + km$ clauses.

Idea: introduce variables to represent boolean statements about colors, and clauses to enforce conditions.
Introduce *indicator variables*

$$x_{v,i} \qquad \text{for } v \in V, \quad i \in [k] = \{0, \ldots, k-1\}.$$

Interpret $x_{v,i} = 1$ as "vertex v receives color i."

Graph Coloring as a SAT Problem

Input: a graph $G = (V, E)$ and a number of colors $k \geq 1$.
Goal: assign colors so that adjacent vertices have different colors.

Theorem

Graph Coloring with k colors on n vertices and m edges reduces in $O(n + km)$ time to CNF-SAT with at most kn variables and $n + km$ clauses.

Idea: introduce variables to represent boolean statements about colors, and clauses to enforce conditions.
Introduce *indicator variables*

$$x_{v,i} \qquad \text{for } v \in V, \quad i \in [k] = \{0, \ldots, k-1\}.$$

Interpret $x_{v,i} = 1$ as "vertex v receives color i."

Q: Which constraints must the CNF formula enforce?

Graph Coloring: The Clauses

1. **Every vertex receives a color.**

Graph Coloring: The Clauses

1. Every vertex receives a color.

$$\left(x_{v,0} \vee x_{v,1} \vee \cdots \vee x_{v,k-1}\right) \qquad \text{for each } v \in V.$$

Graph Coloring: The Clauses

1. Every vertex receives a color.

$$\left(x_{v,0} \vee x_{v,1} \vee \cdots \vee x_{v,k-1}\right) \qquad \text{for each } v \in V.$$

2. Adjacent vertices cannot share a color.

Graph Coloring: The Clauses

1. Every vertex receives a color.

$$\left(x_{v,0} \vee x_{v,1} \vee \cdots \vee x_{v,k-1}\right) \qquad \text{for each } v \in V.$$

2. Adjacent vertices cannot share a color.

$$\left(\neg x_{u,i} \vee \neg x_{v,i}\right) \qquad \text{for each } \{u, v\} \in E,\ i \in [k].$$

Graph Coloring: The Clauses

1. Every vertex receives a color.

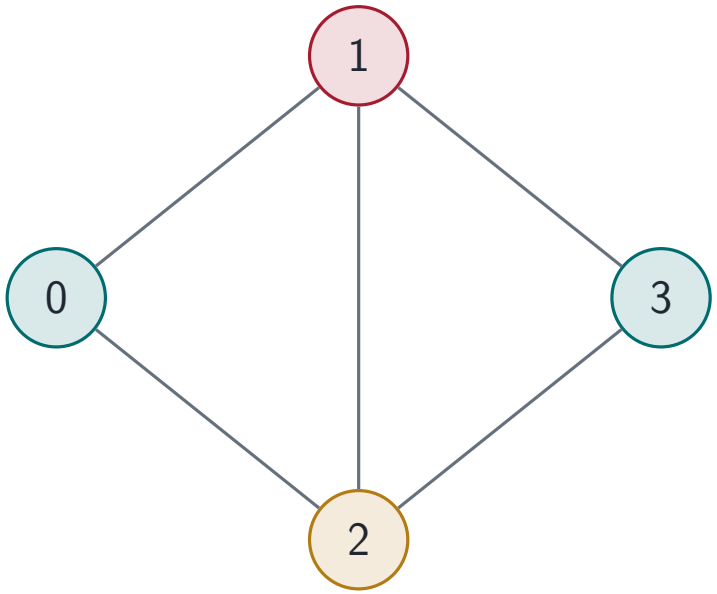
$$\left(x_{v,0} \vee x_{v,1} \vee \cdots \vee x_{v,k-1}\right) \qquad \text{for each } v \in V.$$

2. Adjacent vertices cannot share a color.

$$\left(\neg x_{u,i} \vee \neg x_{v,i}\right) \qquad \text{for each } \{u, v\} \in E,\ i \in [k].$$

Let ϕ_G be the AND of all these clauses.

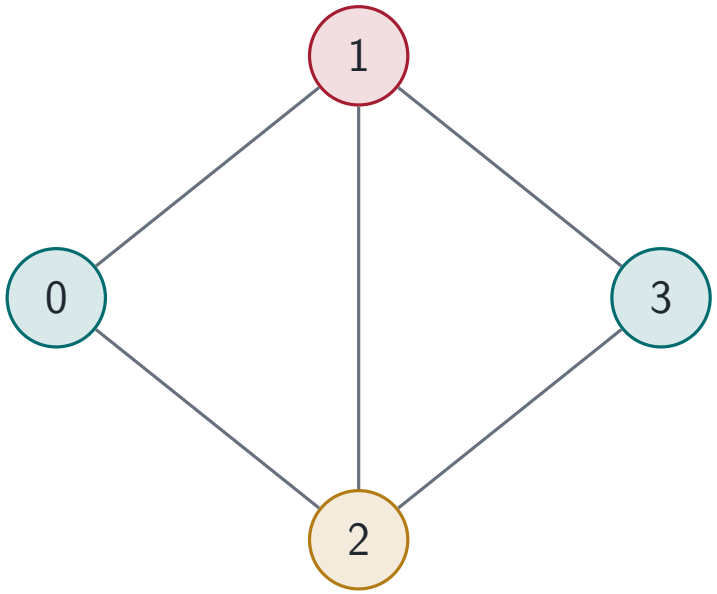
Graph Coloring: Example



Color 0 Color 1 Color 2

Here $n = 4$, $m = 5$, and $k = 3$.

Graph Coloring: Example

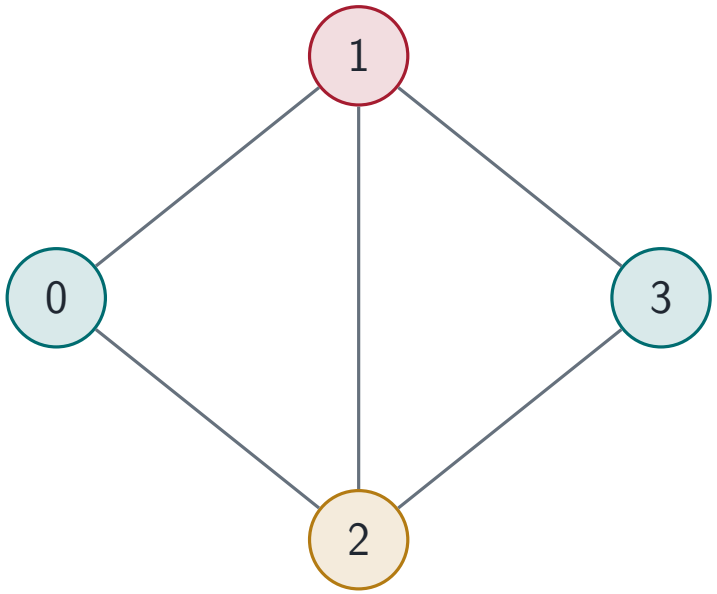


Here $n = 4$, $m = 5$, and $k = 3$.

For vertex 0, include

$$(x_{0,0} \vee x_{0,1} \vee x_{0,2}).$$

Graph Coloring: Example



Here $n = 4$, $m = 5$, and $k = 3$.

For vertex 0, include

$$(x_{0,0} \vee x_{0,1} \vee x_{0,2}).$$

For edge $\{0, 1\}$, include

$$(\neg x_{0,0} \vee \neg x_{1,0}),$$
$$(\neg x_{0,1} \vee \neg x_{1,1}),$$
$$(\neg x_{0,2} \vee \neg x_{1,2}).$$

Graph Coloring: Correctness

Claim 1. A proper k-coloring f gives a satisfying assignment.

Graph Coloring: Correctness

Claim 1. A proper k-coloring f gives a satisfying assignment.

$$\alpha_{v,i} = 1 \quad \Longleftrightarrow \quad f(v) = i.$$

Every vertex has a color, and no edge has two endpoints of the same color.

Graph Coloring: Correctness

Claim 1. A proper k-coloring f gives a satisfying assignment.

$$\alpha_{v,i} = 1 \quad \Longleftrightarrow \quad f(v) = i.$$

Every vertex has a color, and no edge has two endpoints of the same color.

Claim 2. A satisfying assignment α gives a proper k-coloring.

$$f_\alpha(v) = \min\{i \in [k] : \alpha_{v,i} = 1\}.$$

Graph Coloring: Correctness

Claim 1. A proper k-coloring f gives a satisfying assignment.

$$\alpha_{v,i} = 1 \quad \Longleftrightarrow \quad f(v) = i.$$

Every vertex has a color, and no edge has two endpoints of the same color.

Claim 2. A satisfying assignment α gives a proper k-coloring.

$$f_\alpha(v) = \min\{i \in [k] : \alpha_{v,i} = 1\}.$$

- Type 1 clauses make $f_\alpha(v)$ well-defined.
- Type 2 clauses prevent adjacent vertices from choosing the same color.

Graph Coloring: Size and Running Time

The direct encoding has

$$\underbrace{kn}_{\text{variables}}, \qquad \underbrace{n + km}_{\text{clauses}}, \qquad \underbrace{kn + 2km}_{\text{literal occurrences}}.$$

Writing it directly takes $O(k(n + m))$ time.

Graph Coloring: Size and Running Time

The direct encoding has

$$\underbrace{kn}_{\text{variables}}, \qquad \underbrace{n + km}_{\text{clauses}}, \qquad \underbrace{kn + 2km}_{\text{literal occurrences}}.$$

Writing it directly takes $O(k(n+m))$ time.

To obtain $O(n+km)$: remove isolated vertices first. The remaining graph has $n' \leq 2m$ vertices, so

$$O(n) + O(kn' + km) = O(n + km).$$

Color the isolated vertices arbitrarily after decoding the SAT solution.

Graph Coloring: Size and Running Time

The direct encoding has

$$\underbrace{kn}_{\text{variables}}, \qquad \underbrace{n+km}_{\text{clauses}}, \qquad \underbrace{kn+2km}_{\text{literal occurrences}}.$$

Writing it directly takes $O(k(n+m))$ time.

To obtain $O(n+km)$: remove isolated vertices first. The remaining graph has $n' \leq 2m$ vertices, so

$$O(n) + O(kn' + km) = O(n+km).$$

Color the isolated vertices arbitrarily after decoding the SAT solution.

Why do this reduction? SAT solvers - powerful heuristic algorithms(may be inefficient sometimes) - can solve many different problems after encoding.

Resolution

Resolution: derive new clauses that every satisfying assignment must obey. It leads to a clean algorithm to solve SAT, although the algorithm may be expensive sometimes.

Resolution

Resolution: derive new clauses that every satisfying assignment must obey. It leads to a clean algorithm to solve SAT, although the algorithm may be expensive sometimes.

Basic idea: given two clauses $C = \ell \vee A$ and $D = \neg\ell \wedge B$ (A, B are clauses themselves), interpret them as $\neg\ell \implies A$ and $\ell \implies B$. Clearly, one of A, B should hold, which means $A \vee B$ holds.

Resolution

Resolution: derive new clauses that every satisfying assignment must obey. It leads to a clean algorithm to solve SAT, although the algorithm may be expensive sometimes.

Basic idea: given two clauses $C = \ell \vee A$ and $D = \neg\ell \wedge B$ (A, B are clauses themselves), interpret them as $\neg\ell \implies A$ and $\ell \implies B$. Clearly, one of A, B should hold, which means $A \vee B$ holds.

A useful clean up to do resolution is `Simplify`:

1. Remove duplicate literals.
2. Return 1 (true) if the clause contains both x_i and $\neg x_i$.
3. Sort the remaining literals by variable index.

Resolution

Resolution: derive new clauses that every satisfying assignment must obey. It leads to a clean algorithm to solve SAT, although the algorithm may be expensive sometimes.

Basic idea: given two clauses $C = \ell \vee A$ and $D = \neg\ell \wedge B$ (A, B are clauses themselves), interpret them as $\neg\ell \implies A$ and $\ell \implies B$. Clearly, one of A, B should hold, which means $A \vee B$ holds.

A useful clean up to do resolution is `Simplify`:

1. Remove duplicate literals.
2. Return 1 (true) if the clause contains both x_i and $\neg x_i$.
3. Sort the remaining literals by variable index.

$$\texttt{Simplify}(x_2 \vee \neg x_0 \vee x_2), \texttt{Simplify}(x_0 \vee \neg x_0 \vee x_1)$$

The Resolution Rule

Suppose C contains a literal ℓ and D contains $\neg\ell$. Their *resolvent* is

$$C \diamond D = \texttt{Simplify}\big((C \setminus \{\ell\}) \vee (D \setminus \{\neg\ell\})\big).$$

The Resolution Rule

Suppose C contains a literal ℓ and D contains $\neg\ell$. Their *resolvent* is

$$C \diamond D = \texttt{Simplify}\big((C \setminus \{\ell\}) \vee (D \setminus \{\neg\ell\})\big).$$

Example:

$$(\neg x_0 \vee x_1) \diamond (\neg x_1 \vee x_2) = (\neg x_0 \vee x_2).$$

If there is no complementary pair, define $C \diamond D = 1$.

Resolution: Examples

Q: Resolve the following clauses, then simplify.

$$(x_0 \vee \neg x_1 \vee x_3 \vee \neg x_5) \\ \diamond (x_1 \vee \neg x_4 \vee \neg x_5).$$

Resolution: Examples

Q: Resolve the following clauses, then simplify.

$$(x_0 \vee \neg x_1 \vee x_3 \vee \neg x_5) \\ \diamond (x_1 \vee \neg x_4 \vee \neg x_5).$$

Q: What if there are two possible complementary pairs?

$$(x_0 \vee x_1 \vee \neg x_4) \diamond (\neg x_0 \vee x_2 \vee x_4).$$

Does the choice of pair change the simplified result?

The Resolution Theorem

View a CNF as a set $\mathcal{C}$ of clauses. An assignment must satisfy them all.

$$(\ell) \diamond (\neg\ell) = (\,) = \text{false}.$$

The *empty clause* contains no literals, so no assignment satisfies it.

Closed under resolution: $C \diamond D \in \mathcal{C} \cup \{1\}$ for every $C, D \in \mathcal{C}$.

Theorem. If $\mathcal{C}$ is closed under resolution, then

- $(\,) \in \mathcal{C}$ if and only if $\mathcal{C}$ is unsatisfiable.
- Otherwise, `ExtractAssignment`$(\mathcal{C})$ (next lecture) finds a satisfying assignment in $O(n + w|\mathcal{C}|)$ time, where w is the maximum clause width.